ChatGPT said:

For a UAV system, use **binary packets over TCP** instead of sending text commands such as `"ARM\n"` or `"STATUS\n"`.

Each packet has a defined structure. The GCS sends command packets, and the UAV simulator sends acknowledgement and telemetry packets back.

1. Packet architecture

GCS — Qt TCP Client
Build command packet → Send → Decode telemetry packet

↑ Command packet ↓ Telemetry packet

UAV — Qt TCP Server
Parse command packet → Update simulated state → Send telemetry

We'll use a simple custom binary protocol for learning. It is **not MAVLink**, but the same general packet-based idea applies.

2. Define the binary packet

We'll use a fixed-size **16-byte packet** so the receiver knows exactly where each packet ends.

Packet layout

Bytes	Field
0–1	Magic: 0x55 0x56
2	Protocol version
3	Packet type
4–5	Sequence number
6–13	Payload (8 bytes)

14–15	Checksum

Packet types:

- `1` — command from GCS
- `2` — telemetry from UAV

Command IDs: `1 = ARM`, `2 = TAKEOFF`, `3 = LAND`, `4 = STATUS`.

Telemetry payload:

Payload bytes	Meaning
0	Armed: 0 or 1
1	Flying: 0 or 1
2–3	Altitude in centimetres
4–5	Battery percentage
6–7	Reserved

All multi-byte values use **big-endian byte order**. The checksum here is a simple 16-bit additive checksum for this educational protocol, not a production-grade CRC.

3. Shared packet code — `packet.h`

Both programs use the same packet definition, encoder, checksum, and decoder.

```
#pragma once

#include <QByteArray>
#include <QtGlobal>

constexpr int PACKET_SIZE = 16;
constexpr quint8 PROTOCOL_VERSION = 1;

enum PacketType : quint8 {
    COMMAND = 1,
    TELEMETRY = 2
};

enum CommandId : quint8 {
```

```
        ARM = 1,
        TAKEOFF = 2,
        LAND = 3,
        STATUS = 4
    };

    inline quint16 readU16(const QByteArray& b, int pos)
    {
        return (quint16(quint8(b[pos])) << 8) |
                quint16(quint8(b[pos + 1]));
    }

    inline void writeU16(QByteArray& b, int pos, quint16 value)
    {
        b[pos]     = char((value >> 8) & 0xFF);
        b[pos + 1] = char(value & 0xFF);
    }

    inline quint16 checksum16(const QByteArray& b)
    {
        quint32 sum = 0;

        for (int i = 0; i < 14; ++i)
            sum += quint8(b[i]);

        return quint16(sum & 0xFFFF);
    }

    inline QByteArray makePacket(
        quint8 type, quint16 sequence, quint8 command = 0,
        bool armed = false, bool flying = false,
        quint16 altitudeCm = 0, quint16 battery = 100)
    {
        QByteArray p(PACKET_SIZE, '\0');

        p[0] = char(0x55);
        p[1] = char(0x56);
        p[2] = char(PROTOCOL_VERSION);
        p[3] = char(type);

        writeU16(p, 4, sequence);

        // Payload
        p[6] = char(command);
        p[7] = char(armed ? 1 : 0);
```

```
    p[8] = char(flying ? 1 : 0);
    writeU16(p, 9, altitudeCm);
    writeU16(p, 11, battery);

    // Bytes 13 is reserved.
    writeU16(p, 14, checksum16(p));

    return p;
}

inline bool validPacket(const QByteArray& p)
{
    return p.size() == PACKET_SIZE &&
           quint8(p[0]) == 0x55 &&
           quint8(p[1]) == 0x56 &&
           quint8(p[2]) == PROTOCOL_VERSION &&
           readU16(p, 14) == checksum16(p);
}
```

4. UAV simulator — `uav_server.cpp`

The UAV receives command packets, updates its simulated state, and sends a binary telemetry packet in response to every valid command.

```
#include <QCoreApplication>
#include <QTcpServer>
#include <QTcpSocket>
#include <QHostAddress>
#include <QDebug>

#include "packet.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    QTcpServer server;
    QTcpSocket* client = nullptr;
    QByteArray rxBuffer;

    bool armed = false;
    bool flying = false;
    quint16 altitudeCm = 0;
```

```
quint16 battery = 100;

if (!server.listen(QHostAddress::LocalHost, 8080)) {
    qCritical() << server.errorString();
    return 1;
}

qInfo() << "UAV simulator listening on TCP port 8080";

QObject::connect(&server, &QTcpServer::newConnection, [&]() {
    while (server.hasPendingConnections()) {
        QTcpSocket* socket = server.nextPendingConnection();

        if (client) {
            socket->disconnectFromHost();
            socket->deleteLater();
            continue;
        }

        client = socket;
        rxBuffer.clear();
        qInfo() << "GCS connected";

        QObject::connect(socket, &QTcpSocket::readyRead,
                         [&, socket]() {
            rxBuffer += socket->readAll();

            while (rxBuffer.size() >= PACKET_SIZE) {
                QByteArray packet = rxBuffer.left(PACKET_SIZE);

                if (!validPacket(packet)) {
                    // Search for the next possible packet boundary.
                    rxBuffer.remove(0, 1);
                    continue;
                }

                rxBuffer.remove(0, PACKET_SIZE);

                if (quint8(packet[3]) != COMMAND)
                    continue;

                quint16 seq = readU16(packet, 4);
                quint8 cmd = quint8(packet[6]);

                qInfo() << "Command ID:" << cmd
```

```
                << "Sequence:" << seq;

        switch (cmd) {
        case ARM:
            if (!flying)
                armed = true;
            break;

        case TAKEOFF:
            if (armed) {
                flying = true;
                altitudeCm = 1000; // Simulated 10 metres
                if (battery > 0)
                    --battery;
            }
            break;

        case LAND:
            altitudeCm = 0;
            flying = false;
            armed = false;
            break;

        case STATUS:
            break;

        default:
            qWarning() << "Unknown command";
            continue;
        }

        // Send current UAV state as a telemetry packet.
        QByteArray telemetry = makePacket(
            TELEMETRY, seq, 0,
            armed, flying, altitudeCm, battery);

        socket->write(telemetry);
    }
});

QObject::connect(socket, &QTcpSocket::disconnected,
                 [&, socket]() {
    qInfo() << "GCS disconnected";

    if (client == socket) {
```

```
                    client = nullptr;
                    rxBuffer.clear();
                }

                socket->deleteLater();
            });
        }
    });

    return app.exec();
}
```

5. Ground Control Station — `gcs_client.cpp`

The GCS sends a 16-byte command packet and decodes the returned telemetry packet.

```
#include <QCoreApplication>
#include <QTcpSocket>
#include <QHostAddress>
#include <QSocketNotifier>
#include <QDebug>

#include <cstdio>
#include <unistd.h>

#include "packet.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    QTcpSocket socket;
    QByteArray rxBuffer;
    quint16 sequence = 1;

    if (!socket.waitForConnected(1)) {
        // The socket is not connected yet; connect below.
    }

    QObject::connect(&socket, &QTcpSocket::readyRead, [&]() {
        rxBuffer += socket.readAll();

        while (rxBuffer.size() >= PACKET_SIZE) {
```

```
        QByteArray packet = rxBuffer.left(PACKET_SIZE);

        if (!validPacket(packet)) {
            rxBuffer.remove(0, 1);
            continue;
        }

        rxBuffer.remove(0, PACKET_SIZE);

        if (quint8(packet[3]) != TELEMETRY)
            continue;

        quint16 seq = readU16(packet, 4);
        bool armed = quint8(packet[7]) != 0;
        bool flying = quint8(packet[8]) != 0;
        quint16 altitudeCm = readU16(packet, 9);
        quint16 battery = readU16(packet, 11);

        qInfo() << "\n--- UAV TELEMETRY ---";
        qInfo() << "Sequence:" << seq;
        qInfo() << "Armed:" << (armed ? "YES" : "NO");
        qInfo() << "Flying:" << (flying ? "YES" : "NO");
        qInfo() << "Altitude:"
                << altitudeCm / 100.0 << "m";
        qInfo() << "Battery:" << battery << "%";
    }
});

QObject::connect(&socket,
                 &QTcpSocket::errorOccurred,
                 [&](QAbstractSocket::SocketError) {
    qWarning() << "TCP error:" << socket.errorString();
});

socket.connectToHost(QHostAddress::LocalHost, 8080);

if (!socket.waitForConnected(3000)) {
    qCritical() << "Cannot connect:" << socket.errorString();
    return 1;
}

qInfo() << "Connected to UAV simulator";

QSocketNotifier input(STDIN_FILENO, QSocketNotifier::Read);
```

```
qInfo().noquote()
    << "\n1 ARM\n2 TAKEOFF\n3 LAND\n4 STATUS\n0 EXIT";

QObject::connect(&input, &QSocketNotifier::activated, [&]() {
    char line[64];

    if (!std::fgets(line, sizeof(line), stdin)) {
        input.setEnabled(false);
        socket.disconnectFromHost();
        app.quit();
        return;
    }

    int choice = -1;
    if (std::sscanf(line, "%d", &choice) != 1) {
        qInfo() << "Enter a menu number";
        return;
    }

    quint8 command = 0;

    switch (choice) {
    case 1: command = ARM; break;
    case 2: command = TAKEOFF; break;
    case 3: command = LAND; break;
    case 4: command = STATUS; break;
    case 0:
        input.setEnabled(false);
        socket.disconnectFromHost();
        app.quit();
        return;
    default:
        qInfo() << "Invalid choice";
        return;
    }

    QByteArray packet = makePacket(COMMAND, sequence++, command);

    qInfo() << "Sending packet:"
            << packet.toHex(' ');

    socket.write(packet);
    socket.flush();
});
```

```
    return app.exec();
}
```

6. Compile and run on Manjaro

Keep all four files in the same directory. In `gcs_client.cpp`, remove this unnecessary block before `connectToHost()`:

```
if (!socket.waitForConnected(1)) {
    // The socket is not connected yet; connect below.
}
```

Install dependencies:

```
sudo pacman -S qt6-base cmake gcc
```

Build both applications:

```
cmake -S . -B build
cmake --build build
```

Open two terminals.

Terminal 1 — UAV simulator

```
./build/uav_server
```

Terminal 2 — GCS

```
./build/gcs_client
```

7. See the binary packets in action

Select `1` to arm, `2` to take off, and `4` to request status.

The GCS will print a hexadecimal packet similar to:

```
Sending packet: 55 56 01 01 00 01 01 00 00 00 00 00 64 00 01 72
```

This example represents a command packet with sequence number `1` and command ID `1` (ARM). The UAV replies with a telemetry packet containing its armed state, flying state, altitude, and battery percentage.

For example:

```
--- UAV TELEMETRY ---
Sequence: 1
Armed: YES
Flying: NO
Altitude: 0 m
Battery: 100 %
```

The sequence number lets the GCS associate telemetry with a command. In a production system, you would also need explicit acknowledgement semantics, timeouts, robust framing and resynchronization, and a stronger integrity check.

Important distinction: this custom packet protocol is only for learning. If your goal is to communicate with a real PX4 or ArduPilot vehicle, use the established MAVLink packet format and command/telemetry message definitions instead of this custom format.